

Python Programming

A Comprehensive Tutorial

Python is a high-level, interpreted, general-purpose programming language known for its clean syntax and readability. Created by Guido van Rossum and first released in 1991, Python has grown to become one of the most popular languages in the world, widely used in web development, data science, automation, artificial intelligence, and more.

1. Variables and Data Types

Python is dynamically typed — you don't need to declare variable types explicitly.

Basic Data Types

Type	Example	Description
int	x = 42	Whole numbers
float	pi = 3.14	Decimal numbers
str	name = "Alice"	Text strings
bool	flag = True	True or False
list	nums = [1, 2, 3]	Ordered, mutable collection
tuple	coords = (10, 20)	Ordered, immutable collection
dict	info = {"a": 1}	Key-value pairs
set	s = {1, 2, 3}	Unique unordered elements

Example Code:

```
# Variable assignment
name = 'Alice'
age = 30
height = 5.7
is_student = False

# Type checking
print(type(name)) # <class 'str'>
print(type(age)) # <class 'int'>

# Type conversion
```

```
num_str = str(age) # '30'  
num_int = int('25') # 25
```

2. Control Flow

Python uses indentation (whitespace) to define code blocks rather than braces.

If / Elif / Else

```
score = 85

if score >= 90:
    grade = 'A'
elif score >= 75:
    grade = 'B'
elif score >= 60:
    grade = 'C'
else:
    grade = 'F'

print(f'Grade: {grade}') # Grade: B
```

Loops

```
# For loop over a list
fruits = ['apple', 'banana', 'cherry']
for fruit in fruits:
    print(fruit)

# For loop with range
for i in range(5): # 0, 1, 2, 3, 4
    print(i)

# While loop
count = 0
while count < 3:
    print(count)
    count += 1
```

3. Functions

Functions allow you to encapsulate reusable blocks of code. In Python, functions are defined using the **def** keyword.

```
# Basic function
def greet(name):
    return f'Hello, {name}!'

# Default arguments
def power(base, exponent=2):
    return base ** exponent

print(power(3)) # 9
print(power(2, 10)) # 1024
```

```
# *args and **kwargs
def summarize(*args, **kwargs):
    print('Args:', args)
    print('Kwargs:', kwargs)

summarize(1, 2, 3, name='Alice', age=30)
```

4. Lists and Comprehensions

Lists are one of the most commonly used data structures in Python. List comprehensions provide a concise way to create lists.

```
# List operations
nums = [3, 1, 4, 1, 5, 9, 2, 6]
nums.append(7) # Add to end
nums.sort() # Sort in place
nums.remove(1) # Remove first '1'
sliced = nums[2:5] # Slicing

# List comprehension
squares = [x**2 for x in range(10)]
# [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]

# Filtered comprehension
evens = [x for x in range(20) if x % 2 == 0]
# [0, 2, 4, 6, 8, 10, 12, 14, 16, 18]
```

5. Object-Oriented Programming

Python is a fully object-oriented language. Classes provide a blueprint for creating objects that bundle data and behaviour together.

```
class Animal:
    def __init__(self, name, species):
        self.name = name
        self.species = species

    def speak(self):
        return f'{self.name} makes a sound.'

class Dog(Animal): # Inheritance
    def speak(self): # Override
        return f'{self.name} says: Woof!'

dog = Dog('Rex', 'Canine')
print(dog.speak()) # Rex says: Woof!
```

6. Useful Built-in Functions

Function	Description	Example
<code>len()</code>	Returns length	<code>len([1,2,3]) → 3</code>
<code>range()</code>	Generates a range of numbers	<code>range(0, 5) → 0..4</code>
<code>print()</code>	Outputs to console	<code>print("Hi")</code>
<code>input()</code>	Reads user input	<code>name = input("Name: ")</code>
<code>int()</code> / <code>float()</code> / <code>str()</code>	Type conversion	<code>int('42') → 42</code>
<code>max()</code> / <code>min()</code>	Largest/smallest value	<code>max(3, 1, 4) → 4</code>
<code>sum()</code>	Sum of iterable	<code>sum([1,2,3]) → 6</code>
<code>sorted()</code>	Returns sorted list	<code>sorted([3,1,2]) → [1,2,3]</code>
<code>enumerate()</code>	Index + value pairs	<code>for i, v in enumerate(lst)</code>
<code>zip()</code>	Pairs up iterables	<code>zip([1,2], ['a','b'])</code>
<code>map()</code>	Applies function to iterable	<code>map(str, [1,2,3])</code>
<code>filter()</code>	Filters by condition	<code>filter(bool, [0,1,2])</code>

7. File Handling

Python makes reading and writing files straightforward using the built-in **open()** function and context managers (the **with** statement).

```
# Writing to a file
with open('data.txt', 'w') as f:
    f.write('Hello, file!\n')

# Reading a file
with open('data.txt', 'r') as f:
    content = f.read()

# Reading line by line
with open('data.txt', 'r') as f:
    for line in f:
        print(line.strip())

# Appending to a file
with open('data.txt', 'a') as f:
    f.write('Another line\n')
```

8. Exception Handling

Python uses try/except blocks to handle errors gracefully without crashing the program.

```
try:
    result = 10 / 0
except ZeroDivisionError as e:
    print(f'Error: {e}')
except (TypeError, ValueError) as e:
    print(f'Type/Value error: {e}')
else:
    print('No error!')
finally:
    print('This always runs')
```

Quick Reference: Python Keywords

Keyword	Purpose
import	Import a module
from	Import specific parts of a module
as	Alias for import or exception
class	Define a class
def	Define a function
return	Return value from function
lambda	Anonymous function
if / elif / else	Conditional branching
for / while	Looping constructs
break / continue	Loop control
try / except / finally	Exception handling
with	Context manager
global / nonlocal	Variable scope control
pass	Placeholder (no-op)
yield	Generator function
assert	Debugging assertion
del	Delete variable or item